

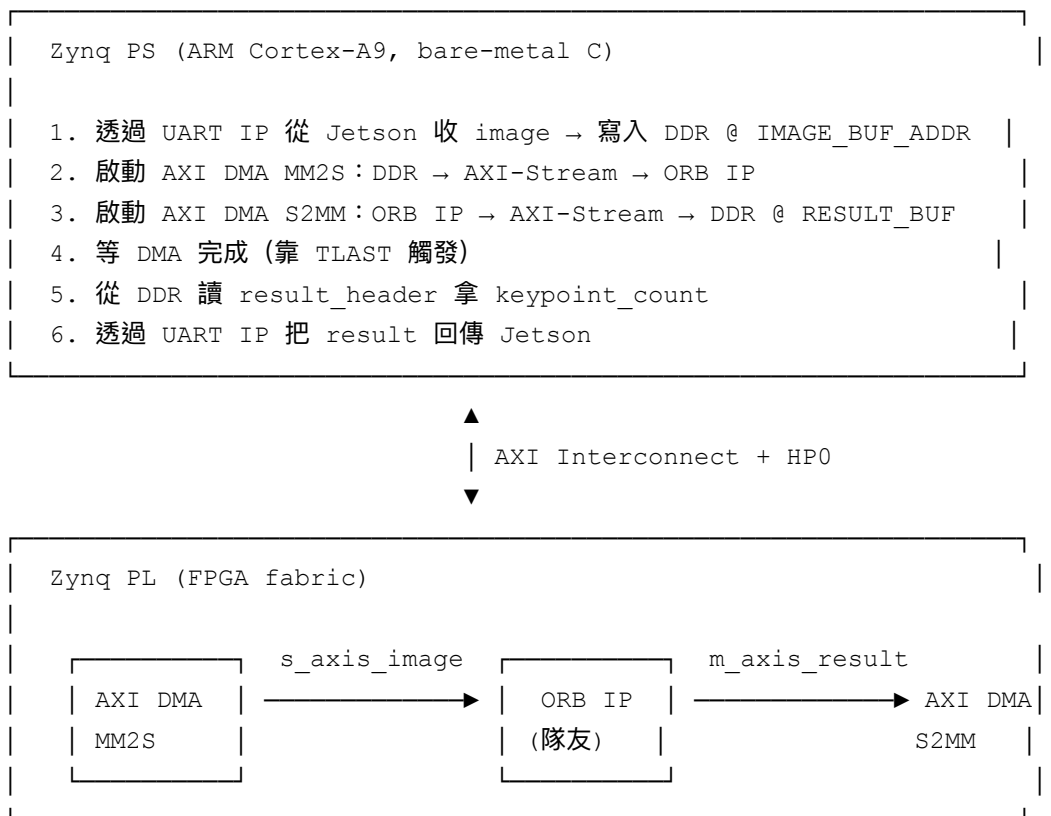
ORB IP 整合介面合約 v1.0

文件目的： 定義 PL 端 ORB IP 與 PS 端整合的硬性規格，確保雙方獨立開發後能無痛整合。

雙方需求：

- PL 端（隊友）：依此規格設計 orb_ip_v1_0 的 AXI-Stream 輸出介面
- PS 端（你）：依此規格寫 C 程式驅動 DMA 並解析 result

1. 系統架構概觀



2. AXI-Stream 介面規格

2.1 輸入：s_axis_image (Image 進 ORB IP)

訊號	寬度	方向	說明
s_axis_image_tdata	32 bit	input	每 beat 4 個 GRAY8 pixel (little-endian packing)
s_axis_image_tvalid	1 bit	input	DMA 有資料時拉高
s_axis_image_tready	1 bit	output	IP 準備好接收時拉高
s_axis_image_tlast	1 bit	input	DMA 在最後一個 beat 拉高

Image 資料順序： Row-major，從 (0,0) 到 (W-1, H-1)，每 4 個 pixel 打包成 1 個 32-bit word。

範例 (80×60 image, 4800 bytes = 1200 words)：

```

Beat 0 : [pix(3)][pix(2)][pix(1)][pix(0)]      TLAST=0
Beat 1 : [pix(7)][pix(6)][pix(5)][pix(4)]      TLAST=0
...
Beat 1199: [pix(4799)]...[pix(4796)]          TLAST=1 ← 最後一拍

```

2.2 輸出：m_axis_result (ORB IP 送 result)

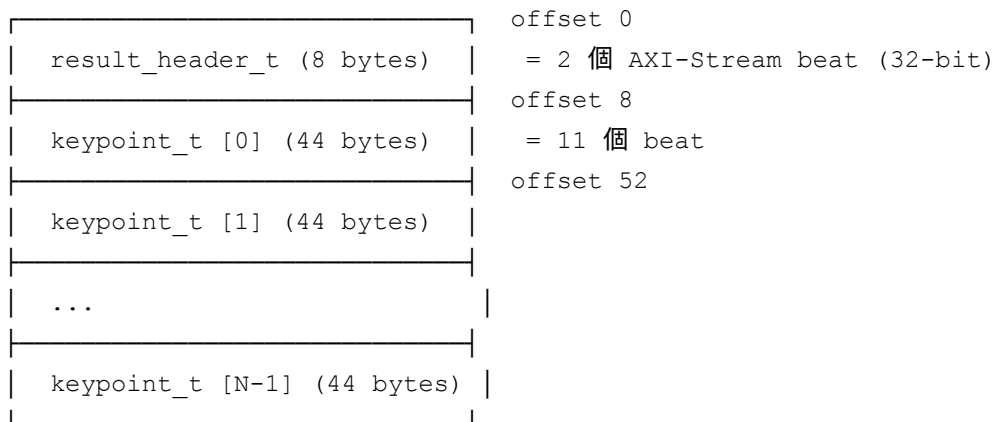
訊號	寬度	方向	說明
m_axis_result_tdata	32 bit	output	Result data
m_axis_result_tvalid	1 bit	output	IP 有資料送時拉高
m_axis_result_tready	1 bit	input	DMA 準備好接收時拉高
m_axis_result_tlast	1 bit	output	在最後一個 word 拉高 ★

★ **TLAST** 是核心：DMA S2MM 靠 TLAST 知道何時結束，沒拉 TLAST 會導致 DMA 永遠等下去。

3. Result 資料格式

3.1 整體佈局

DDR Result Buffer (@ 0x10100000):



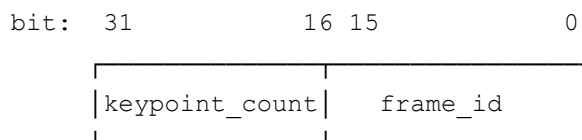
總 byte 數 = 8 + N × 44 bytes

總 word 數 = 2 + N × 11 words

3.2 result_header_t (8 bytes / 2 words)

```
typedef struct __attribute__((packed)) {
    uint16_t frame_id;          // offset 0, bit [15:0] of word 0
    uint16_t keypoint_count;    // offset 2, bit [31:16] of word 0
    uint32_t reserved;         // offset 4, word 1 (保留 + 對齊)
} result_header_t;
```

Word 0 (little-endian 32-bit) :



frame_id : PL 端可以維持一個 counter，每 frame +1。或者從 ORB IP 的 AXI-Lite 暫存器設定（看你的 IP 設計）。

keypoint_count : 這張 frame 偵測到的 keypoint 數量，必須準確，PS 靠這個算總 byte 數。

3.3 keypoint_t (44 bytes / 11 words)

```
typedef struct __attribute__((packed)) {
    int16_t x;                  // offset 0 (signed pixel coord)
    int16_t y;                  // offset 2
    uint16_t angle;             // offset 4 (16-bit fixed-point, 0-65535 → 0-360°)
    uint8_t octave;             // offset 6 (nLevels=1 時固定 0)
    uint8_t padding;            // offset 7 (對齊用，固定 0)
```

```

uint32_t response;          // offset 8    (Harris score)
uint8_t  descriptor[32];    // offset 12   (BRIEF 256-bit)
} keypoint_t;

// 編譯時靜態檢查
_Static_assert(sizeof(keypoint_t) == 44, "keypoint_t must be 44 bytes");

```

Word-by-word breakdown :

Word	Bytes	內容
0	0-3	[y:16][x:16]
1	4-7	[padding:8][octave:8][angle:16]
2	8-11	response (uint32)
3-10	12-43	descriptor[0..31] (8 個 word)

4. TLAST 規則（最關鍵）

4.1 必須在最後一個 word 拉 TLAST 一拍

假設 $N = 5$ 個 keypoint :

總 word 數 = 2 (header) + 5 × 11 (keypoints) = 57 words

時序：

```

Word 0:  TVALID=1, TLAST=0  ← header word 0
Word 1:  TVALID=1, TLAST=0  ← header word 1
Word 2:  TVALID=1, TLAST=0  ← keypoint[0] word 0
Word 3:  TVALID=1, TLAST=0  ← keypoint[0] word 1
...
Word 56: TVALID=1, TLAST=1  ← keypoint[4] 的最後一個 word ★

```

4.2 沒有 keypoint 時也要送

即使 `keypoint_count = 0`，仍要送 header 並在 word 1 拉 TLAST：

```

Word 0: header word 0 (keypoint_count = 0)  TLAST=0
Word 1: header word 1                        TLAST=1  ← 即使沒 keypoint，也要結束

```

4.3 TLAST 拉太早或太晚的後果

情況	後果
沒拉 TLAST	DMA S2MM 永遠 busy，PS hang
TLAST 拉太早	PS 收到不完整 result，keypoint 被截斷
TLAST 拉超過一拍	DMA 可能誤判為下一筆 transfer

正確：在最後一個有效 word 同時拉 TVALID + TLAST 一拍

5. PS 端讀取流程

```
// PS 端 C 程式邏輯 (精簡版)

// 1. 啟動 S2MM DMA，給上限 buffer (譬如 1 MB)
XAxiDma_SimpleTransfer(&dma, RESULT_BUF_ADDR, 1024*1024,
                       XAXIDMA_DEVICE_TO_DMA);

// 2. 啟動 MM2S DMA 把 image 送進 ORB IP
XAxiDma_SimpleTransfer(&dma, IMAGE_BUF_ADDR, IMAGE_BYTES,
                       XAXIDMA_DMA_TO_DEVICE);

// 3. 等兩邊 DMA 完成
while (XAxiDma_Busy(&dma, XAXIDMA_DMA_TO_DEVICE));
while (XAxiDma_Busy(&dma, XAXIDMA_DEVICE_TO_DMA));
// ↑ S2MM 在 ORB IP 拉 TLAST 後才會結束

// 4. Invalidate cache，CPU 才能讀到 DMA 寫的最新資料
Xil_DCacheInvalidateRange((INTPTR)RESULT_BUF_ADDR, 1024*1024);

// 5. 從 header 讀 keypoint_count
volatile result_header_t *hdr = (volatile result_header_t *)RESULT_BUF_ADDR;
uint16_t count = hdr->keypoint_count;
uint32_t total_bytes = sizeof(result_header_t) + count * sizeof(keypoint_t);

// 6. 透過 UART 把 result 送回 Jetson
uart_send_byte(0x55); // magic
uart_send_buffer((uint8_t*)RESULT_BUF_ADDR, total_bytes);
```

6. 整合測試 Checklist

Phase A：隊友單元測試（不需要 PS）

- ORB IP 在模擬器跑通 testbench
- 用 testbench 餵 4800 byte image (80×60)
- 確認 `m_axis_result` 輸出符合 `result_header + keypoints` 格式
- 確認最後一個 word 拉 TLAST 一拍
- 確認沒有 keypoint 時也正確送 header + TLAST

Phase B：整合（你 + 隊友）

- 把 ORB IP 加入你的 BD（取代 dummy IP）
- AXI-Stream 接線確認（width、TLAST、TREADY）
- Generate Bitstream 成功
- PS 端 `USE_REAL_ORB_IP = 1` 編譯成功

Phase C：端到端驗證

- 跑單張 image，PS 端不 hang（代表 TLAST 有拉）
 - `keypoint_count` 在合理範圍（10-200 之間）
 - keypoint 座標 (x, y) 都在 $[0, 80] \times [0, 60]$ 之間
 - 連續跑 10 frame 不丟失
 - Jetson 端視覺化 keypoint 位置和影像內容對應
-

7. 故障排除

症狀 1：PS hang 在 `XAxiDma_Busy()` 永遠不結束

原因：ORB IP 沒拉 TLAST，或拉的時機不對

檢查：用 ILA 抓 `m_axis_result_tlast` 訊號，看有沒有上升沿

症狀 2：`keypoint_count` 數值異常大（譬如 0xDEAD）

原因：ORB IP 沒按 header 格式輸出，PS 把 dummy data 當成 header

檢查： 在 PS 端加 `xil_printf("hdr.frame_id=%u, count=%u\n", hdr->frame_id, count);`

症狀 3：keypoint 座標超出範圍

原因： ORB IP 內部座標計算錯誤，或 endianness 反了

檢查： 在 PS 端印前 5 個 keypoint 的 raw bytes

症狀 4：DMA 報 error 15 (XST_INVALID_PARAM)

原因： transfer length 超過 DMA buffer width 上限

檢查： 確認 BD 中 axi_dma_0 的 “Width of Buffer Length Register” = 23

8. 修訂歷史

Version	Date	Author	Changes
1.0	2026-01-XX	初版	基於 dummy IP 驗證的端到端架構

9. 簽收

- **PL 端（隊友）：** 確認 IP 設計符合本合約 § 2-4
- **PS 端（你）：** 確認 C 程式符合本合約 § 5
- 雙方同意 § 6 的整合測試流程